

Technical Interview Knowledge Guide

Focused preparation for technical questions and follow-up depth

Technical questions and explanations only. No career-story, recruiter, or motivation sections.

TypeScript - React - Node.js - REST - AWS - CI/CD -
Observability - LLM - MCP - AI agents
Phone-friendly edition - July 2026

Contents

1. TypeScript	6 topics
2. React	11 topics
3. Node.js	7 topics
4. REST, HTTP, and API design	6 topics
5. AWS and cloud architecture	6 topics
6. Testing, CI/CD, and observability	4 topics
7. LLMs, MCP, and AI agents	5 topics
8. System design drills	4 topics
9. Live coding and code review	5 topics

1. TypeScript

The role explicitly asks for strong TypeScript. The important part is not syntax, but using the type system to make boundaries, state transitions, and refactoring safer.

What is the practical value of TypeScript in a large application?

CORE ANSWER

TypeScript moves many integration mistakes from runtime to development time. Its strongest value is shared contracts: component props, API responses, domain models, events, and configuration can all be checked consistently. It also makes refactoring safer because the compiler identifies affected call sites.

IMPORTANT DETAILS

- Types do not replace runtime validation. Data from HTTP, storage, queues, and users is still untrusted.
- Prefer domain-specific types over large generic objects with many optional fields.
- A strict configuration gives much more value than using TypeScript as lightly annotated JavaScript.

LIKELY FOLLOW-UPS

- Which strict compiler options matter most?
- Where should runtime validation happen?
- When can types become too complex?

Explain any, unknown, never, and void.

CORE ANSWER

any disables type checking and lets unsafe values flow through the program. unknown also accepts any value, but requires narrowing before use, so it is the correct type for untrusted input. never represents a value that cannot exist, which is useful for exhaustive checks. void means a function's return value is intentionally ignored or absent.

IMPORTANT DETAILS

- Use unknown at API and error boundaries.
- Keep any isolated when it is unavoidable, for example around an incomplete third-party definition.
- Use never to make missing union cases fail at compile time.

LIKELY FOLLOW-UPS

- Why is catch error often unknown?
- What is the difference between never and a function that returns undefined?

How do discriminated unions improve domain modelling?

CORE ANSWER

A discriminated union represents a finite set of valid states using a shared tag such as status or type. Each state carries only the fields that are valid for that case. This prevents impossible combinations such as `loading=true` together with both data and error.

```
type Result<T> =  
  | { status: 'ok'; data: T }  
  | { status: 'error'; error: Error };  
  
function unwrap<T>(r: Result<T>): T {  
  switch (r.status) {  
    case 'ok': return r.data;  
    case 'error': throw r.error;  
    default: return assertNever(r);  
  }  
}
```

IMPORTANT DETAILS

- They work well for request state, workflow state, events, and reducer actions.
- A switch on the discriminator gives precise narrowing.
- An exhaustive default using `never` detects newly added cases.

LIKELY FOLLOW-UPS

- How would you model an order lifecycle?
- Why can a single interface with many optional fields be dangerous?

What makes a good generic abstraction?

CORE ANSWER

A useful generic captures a relationship between values, not just a placeholder type. For example, a function may accept a key that must exist on an object and return the corresponding property type. Constraints should express the minimum capability the implementation needs.

IMPORTANT DETAILS

- Prefer inference so callers do not have to specify type arguments manually.
- Avoid generics that only rename unknown or hide a cast.
- Do not expose implementation-specific generic complexity in a public API unless it gives real safety.

LIKELY FOLLOW-UPS

- What is the difference between `K extends keyof T` and `string`?
- When are overloads clearer than conditional types?

How should API responses be typed safely?

CORE ANSWER

Define a transport schema, validate the raw response at runtime, then map it into a domain model. A TypeScript interface alone only describes what the developer expects; it does not prove that the server actually returned it. Validation libraries or generated clients can turn external data into a trusted internal value.

IMPORTANT DETAILS

- Validate at the boundary once, not repeatedly throughout the application.
- Separate DTOs from domain objects when dates, money, identifiers, or nullable fields need conversion.
- Preserve validation errors with enough context for logging and diagnosis.

LIKELY FOLLOW-UPS

- Would you share types directly between frontend and backend?
- How do OpenAPI-generated types help and where can they fail?

Which tsconfig settings are especially important for production code?

CORE ANSWER

strict is the baseline because it enables the family of strict checks. noUncheckedIndexedAccess catches unsafe indexing, exactOptionalPropertyTypes distinguishes a missing property from an explicit undefined, and useUnknownInCatchVariables improves error handling. noImplicitOverride and noFallthroughCasesInSwitches prevent common maintenance mistakes.

IMPORTANT DETAILS

- Adopt stricter flags incrementally in a legacy codebase.
- Keep build output and type checking separate when the bundler transpiles TypeScript.
- Run type checking in CI even when local tooling performs fast transpilation only.

LIKELY FOLLOW-UPS

- What is the difference between transpilation and type checking?
- How would you migrate a JavaScript project gradually?

2. React

Expect questions about rendering, state ownership, effects, performance, and production behaviour rather than only hook syntax.

What causes a React component to render again?

CORE ANSWER

A component renders when its own state changes, when its parent renders, when a consumed context value changes, or when an external store subscription reports a change. Rendering means React calls the component to calculate the next element tree. It does not automatically mean that the DOM changes.

IMPORTANT DETAILS

- React compares the new tree with the previous one during reconciliation.
- A child normally renders with its parent even when props appear equal, unless memoization prevents it.
- Development Strict Mode may intentionally call render-related logic more than once to expose unsafe side effects.

LIKELY FOLLOW-UPS

- What is the difference between render and commit?
- Does setState with the same value always cause a commit?

How does reconciliation work, and why are keys important?

CORE ANSWER

Reconciliation matches elements between renders by type and position. In a list, keys provide stable identity so React can preserve the correct component state when items move, are inserted, or are removed. A key is not passed as a normal prop.

IMPORTANT DETAILS

- Use a stable domain identifier, not the array index when order can change.
- Changing a key intentionally resets the component subtree.
- Duplicate or unstable keys can attach local state to the wrong item.

LIKELY FOLLOW-UPS

- When is an array index an acceptable key?
- How can a key be used to reset a form?

When do memo, useMemo, and useCallback help?

CORE ANSWER

Memoization helps when a render or calculation is measurably expensive and its inputs are stable. React.memo can skip a child render when props are equal. useMemo caches a calculated value, while useCallback caches a function identity. None of them makes code automatically faster.

IMPORTANT DETAILS

- Memoization itself costs comparisons, memory, and complexity.
- A new object or callback passed from the parent can defeat React.memo.
- Profile first; often moving state closer to where it is used is a better optimization.

LIKELY FOLLOW-UPS

- What does referential equality mean?
- Why can excessive useCallback usage make code worse?

How should state be divided between local state, context, a client store, and server-state tools?

CORE ANSWER

Keep state as local as possible. Use context for relatively stable values needed across a subtree, such as theme or an authenticated user summary. Use a client store for complex shared client-side workflows or cross-cutting state. Treat remotely owned data as server state and use a cache that understands freshness, refetching, deduplication, and invalidation.

IMPORTANT DETAILS

- Do not copy server data into a client store without a clear reason.
- Separate persisted business state from temporary UI state.
- A store solves coordination, not poor domain boundaries.

LIKELY FOLLOW-UPS

- When does context cause performance problems?
- What state should be encoded in the URL?

What are the main useEffect mistakes?

CORE ANSWER

An effect should synchronize React with an external system, such as a subscription, timer, network request, or imperative browser API. Common mistakes are using effects for derived state, omitting dependencies, creating loops, and forgetting cleanup. The effect must be correct when it is started, cleaned up, and started again.

```
useEffect(() => {  
  const controller = new AbortController();  
  loadUser(id,  
    controller.signal).then(setUser).catch(handleError);  
  return () => controller.abort();  
}, [id]);
```

IMPORTANT DETAILS

- Compute derived values during render instead of storing them.
- Abort or ignore obsolete requests to prevent race conditions.
- Dependencies should reflect every reactive value used by the effect.

LIKELY FOLLOW-UPS

- How do stale closures happen?
- Why does Strict Mode run effect setup and cleanup twice in development?

How do stale closures occur in React?

CORE ANSWER

A function created during a render captures the props and state from that render. If it is used later by a timer, subscription, or async callback, it may read an old value. The fix depends on intent: include the value in dependencies, use a functional state update, move mutable non-render data to a ref, or recreate the subscription correctly.

IMPORTANT DETAILS

- Do not silence the dependency linter without understanding the data flow.
- Refs avoid rerenders but also bypass React's reactive model.
- Functional updates are ideal when the next state depends only on the previous state.

LIKELY FOLLOW-UPS

- When is useRef appropriate?
- How would you implement an interval that always sees current state?

How would you prevent request races in a data-fetching component?

CORE ANSWER

A later request should own the result. Cancel the previous request with `AbortController` when possible, or track a request identifier and ignore stale responses. A server-state library usually handles cancellation, deduplication, retries, and cache keys more consistently than custom effects.

IMPORTANT DETAILS

- Include all request inputs in the cache key.
- Distinguish cancellation from a real failure in the UI and logs.
- Do not allow a slow previous response to overwrite newer data.

LIKELY FOLLOW-UPS

- How would you handle optimistic updates?
- What should happen when two mutations conflict?

What are Suspense and transitions useful for?

CORE ANSWER

Suspense lets a subtree declare that it is waiting for a supported asynchronous resource and display a fallback boundary. A transition marks an update as non-urgent, allowing React to keep urgent interactions responsive while preparing the next UI. They improve coordination and perceived responsiveness, but they do not replace caching or error handling.

IMPORTANT DETAILS

- Place boundaries so one slow area does not blank the whole page.
- Use transitions for navigation, filtering, or expensive non-urgent view changes.
- Pair Suspense with error boundaries and a data layer that supports it.

LIKELY FOLLOW-UPS

- What is the difference between a loading boolean and Suspense?
- When would useDeferredValue help?

How do you diagnose a slow React screen?

CORE ANSWER

Start with measurement. Use the React Profiler to find components with expensive or repeated renders, then inspect why they render. Check browser performance traces for long tasks, layout work, network waterfalls, large bundles, and memory growth. Optimize the largest verified bottleneck rather than applying memoization everywhere.

IMPORTANT DETAILS

- Virtualize long lists.
- Split large bundles and lazy-load non-critical routes or features.
- Reduce state fan-out and expensive work during render.
- Check whether the backend or network, rather than React, is the real bottleneck.

LIKELY FOLLOW-UPS

- What are Core Web Vitals?
- How would you detect a memory leak caused by subscriptions?

What do error boundaries catch, and what do they not catch?

CORE ANSWER

An error boundary catches rendering, lifecycle, and constructor errors in its descendant component tree and can show a fallback UI. It does not automatically catch event-handler errors, asynchronous callback errors, server errors, or errors thrown inside the boundary itself.

IMPORTANT DETAILS

- Use boundaries around meaningful product areas, not only once at the application root.
- Report the error with route, release, user context, and correlation identifiers.
- Async failures should be represented in state or surfaced through the data layer.

LIKELY FOLLOW-UPS

- How would you recover from a failed lazy-loaded chunk?
- What information should be sent to an error-monitoring tool?

How would you design a maintainable component API?

CORE ANSWER

Prefer a small, explicit API based on the component's responsibility. Use composition when callers need structural flexibility. Avoid many boolean props that create invalid combinations; variants or discriminated unions are often clearer. Separate data orchestration from reusable presentation when that improves testing and ownership.

IMPORTANT DETAILS

- Keep accessibility semantics inside the reusable component.
- Forward only the DOM props that are intentionally supported.
- Do not create abstractions before multiple real use cases reveal the common shape.

LIKELY FOLLOW-UPS

- When is a compound-component pattern useful?
- How would you test behaviour without coupling tests to implementation details?

3. Node.js

The likely depth areas are the event loop, asynchronous failure handling, scaling, streams, and operational behaviour.

How does the Node.js event loop affect application design?

CORE ANSWER

JavaScript callbacks run on a single main thread, while many I/O operations are delegated to the operating system or libuv. When I/O completes, callbacks are scheduled through event-loop phases. Promise callbacks and other microtasks run with high priority between turns, so a long microtask chain can also delay timers and I/O.

IMPORTANT DETAILS

- Do not block the main thread with heavy CPU work or synchronous APIs on request paths.
- Concurrency is efficient for I/O, but JavaScript execution is still serialized on each process.
- Measure event-loop delay to detect saturation.

LIKELY FOLLOW-UPS

- What is the difference between `process.nextTick` and a Promise microtask?
- Why can JSON parsing become a performance issue?

What is the difference between concurrency and parallelism in Node.js?

CORE ANSWER

Concurrency means multiple operations are in progress, usually while waiting for I/O. Parallelism means work executes simultaneously on multiple CPU cores or processes. `Promise.all` creates concurrency but does not make CPU-heavy JavaScript parallel. Worker threads, multiple processes, or external workers are required for true parallel CPU work.

IMPORTANT DETAILS

- Unbounded `Promise.all` can overload a database or external API.
- Use a concurrency limit when processing large collections.
- Keep request latency predictable by moving long work to queues.

LIKELY FOLLOW-UPS

- When would you use worker threads instead of child processes?
- How would you implement bounded concurrency?

What are streams and backpressure?

CORE ANSWER

Streams process data incrementally instead of buffering the entire payload in memory. Backpressure is the mechanism that slows the producer when the consumer cannot keep up. In Node.js, pipe and pipeline coordinate this flow and propagate completion and errors.

IMPORTANT DETAILS

- Use streams for large uploads, downloads, compression, parsing, and proxying.
- Respect the return value of `writable.write` when implementing manual flow control.
- Use pipeline to avoid missed errors and incomplete cleanup.

LIKELY FOLLOW-UPS

- What happens if backpressure is ignored?
- How would you stream a large database export safely?

How should errors be handled in an async Node.js service?

CORE ANSWER

Handle expected domain and validation failures explicitly, translate them at the transport boundary, and preserve the original cause for logging. Unexpected errors should be captured centrally, correlated with the request, and allowed to trigger a controlled process restart when the process may be inconsistent.

IMPORTANT DETAILS

- Never continue silently after an unhandled rejection or uncaught exception.
- Operational errors such as timeouts are different from programmer errors.
- Use structured error objects with a stable code, safe public message, cause, and metadata.

LIKELY FOLLOW-UPS

- Should an uncaught exception terminate the process?
- How do you prevent sensitive details from leaking in an API response?

How do you scale a stateful WebSocket service horizontally?

CORE ANSWER

Connections remain attached to individual instances, so cross-instance messages require a shared broker or pub/sub layer. Authentication and connection metadata must be handled consistently, and reconnects must tolerate a client landing on another instance. Sticky sessions can reduce movement but do not remove the need for distributed state design.

IMPORTANT DETAILS

- Keep durable business state outside the WebSocket process.
- Define delivery semantics: at-most-once, at-least-once, or application-level acknowledgement.
- Use heartbeats, idle timeouts, connection limits, and backpressure.
- Plan for reconnect storms during deployment or network recovery.

LIKELY FOLLOW-UPS

- How would you avoid duplicate messages?
- What metrics are important for a WebSocket fleet?

What does graceful shutdown require?

CORE ANSWER

On a termination signal, the service should stop accepting new traffic, mark itself unready, allow in-flight requests to finish within a deadline, close broker and database connections, flush essential telemetry, and then exit. Long-lived connections need an explicit drain or reconnect strategy.

IMPORTANT DETAILS

- The orchestrator's termination grace period must be longer than the application's drain deadline.
- Shutdown must be idempotent because multiple signals can arrive.
- Do not wait forever; force termination after the deadline.

LIKELY FOLLOW-UPS

- How would you drain WebSocket clients?
- What is the difference between liveness and readiness checks?

How do you protect a Node.js API from overload?

CORE ANSWER

Apply limits at several layers: request body size, timeouts, rate limits, connection limits, bounded queues, database pool limits, and controlled concurrency for downstream calls. Reject excess work early with a clear response instead of allowing latency and memory usage to grow without bound.

IMPORTANT DETAILS

- Use per-tenant or per-credential limits when fairness matters.
- Circuit breakers protect against repeatedly calling a failing dependency.
- Bulkheads keep one dependency or workload from consuming all resources.
- Monitor queue depth, event-loop delay, memory, and downstream latency.

LIKELY FOLLOW-UPS

- What is load shedding?
- How would you choose timeout values between services?

4. REST, HTTP, and API design

The position includes REST APIs and business-facing features. Be ready to explain semantics, compatibility, security, and distributed failure handling.

What makes an API RESTful?

CORE ANSWER

REST is an architectural style built around resources, a uniform interface, stateless requests, cacheable responses where appropriate, and layered components. In practice, a good HTTP API uses resource-oriented URLs, standard method semantics, meaningful status codes, and representations that can evolve without exposing internal storage details.

IMPORTANT DETAILS

- REST is not simply JSON over HTTP.
- A resource is a business concept, not necessarily a database table.
- Stateless means each request contains the context needed to process it; it does not forbid server-side data.

LIKELY FOLLOW-UPS

- What is the difference between REST and RPC?
- When is an action endpoint acceptable?

Explain safety and idempotency of HTTP methods.

CORE ANSWER

A safe method should not intentionally change server state; GET and HEAD are safe. An idempotent method can be repeated with the same intended effect; PUT and DELETE are idempotent by definition, while POST usually is not. Real implementations must still account for side effects such as logs, notifications, or duplicated downstream operations.

IMPORTANT DETAILS

- Use idempotency keys for retried creation or payment-like operations.
- PATCH may be idempotent depending on the patch format and operation.
- Clients and proxies rely on correct method semantics for retries and caching.

LIKELY FOLLOW-UPS

- How would you implement an idempotency-key store?
- Can DELETE return 404 on the second request and still be idempotent?

How should API errors be designed?

CORE ANSWER

Return a stable machine-readable error code, a safe human-readable message, relevant field-level details, and a correlation identifier. Map categories consistently: invalid input, authentication failure, authorization failure, missing resource, conflict, rate limit, dependency failure, and unexpected server error.

IMPORTANT DETAILS

- Do not expose stack traces or internal SQL and service details.
- A 4xx response means the client must change something; a 5xx response indicates the server or dependency failed.
- Use 409 for a current-state conflict and 422 when syntactically valid input violates semantic validation, if that convention is used consistently.

LIKELY FOLLOW-UPS

- When would you return 400 versus 422?
- How should clients decide whether to retry?

How do you design pagination for large, changing datasets?

CORE ANSWER

Offset pagination is simple and supports jumping to a page, but it becomes expensive at large offsets and can skip or duplicate items when data changes. Cursor pagination uses a stable ordered key and is usually better for feeds and operational datasets. The ordering must be deterministic, commonly using a timestamp plus a unique identifier.

IMPORTANT DETAILS

- Treat cursors as opaque tokens.
- Define the result order explicitly.
- Return a next cursor rather than exposing database internals.
- Filtering and authorization must be included in the cursor's logical context.

LIKELY FOLLOW-UPS

- How do you paginate when multiple rows have the same timestamp?
- What are the trade-offs of total counts?

How do you evolve an API without breaking clients?

CORE ANSWER

Prefer additive changes: new optional fields, new endpoints, or new capabilities that old clients can ignore. Breaking changes include removing or renaming fields, changing meaning or type, making an optional field required, and changing validation or ordering assumptions. Version only when compatibility cannot be preserved.

IMPORTANT DETAILS

- An added enum value can break clients that assume exhaustive handling.
- Use deprecation telemetry and a migration window before removal.
- Contract tests and schema-diff checks help detect accidental breaking changes.

LIKELY FOLLOW-UPS

- Is adding a response field always safe?
- What are the trade-offs of URL, header, and media-type versioning?

How do authentication and authorization differ?

CORE ANSWER

Authentication establishes who or what the caller is. Authorization decides whether that identity may perform the requested action on the specific resource. Authorization must be enforced on the server for every relevant operation, ideally close to the business rule rather than only at a route or UI layer.

IMPORTANT DETAILS

- Use short-lived credentials and validate issuer, audience, signature, expiry, and intended token type.
- Prevent insecure direct object references by checking resource ownership or policy.
- CORS controls browser access; it is not an authorization mechanism.
- For cookie-based authentication, address CSRF explicitly.

LIKELY FOLLOW-UPS

- What is the difference between roles and permissions?
- Where should tenant isolation be enforced?

5. AWS and cloud architecture

The role works in AWS. The strongest answers connect services to availability, security, cost, and operational trade-offs.

Design a highly available web application on AWS.

CORE ANSWER

A common design uses Route 53 and CloudFront at the edge, an Application Load Balancer across multiple Availability Zones, and stateless application instances in private subnets. Data uses a managed multi-AZ database or a suitable distributed store. Static assets live in S3. Autoscaling, health checks, centralized logs, metrics, traces, backups, and infrastructure as code complete the design.

IMPORTANT DETAILS

- Only public-facing load balancers and controlled ingress belong in public subnets.
- Application instances should use IAM roles rather than static credentials.
- High availability requires removing single-instance dependencies and testing failure recovery.

LIKELY FOLLOW-UPS

- Where would you terminate TLS?
- How would you support zero-downtime deployment?
- What changes for multi-region recovery?

When would you choose Lambda instead of containers?

CORE ANSWER

Lambda is strong for event-driven, bursty, short-lived work where minimal infrastructure management is valuable. Containers are better for long-running services, specialized runtimes, predictable high throughput, persistent connections, or workloads needing more control over CPU, memory, networking, and startup behaviour.

IMPORTANT DETAILS

- Compare total cost at the expected traffic pattern, not only unit price.
- Cold starts matter mainly for latency-sensitive and infrequently invoked paths.
- Lambda concurrency can overload downstream systems unless limited.
- Containers still benefit from managed orchestration such as ECS or EKS.

LIKELY FOLLOW-UPS

- How do provisioned concurrency and reserved concurrency differ?
- Would you use Lambda for WebSockets?

How do SQS, SNS, and EventBridge differ?

CORE ANSWER

SQS is a durable queue used to decouple producers and workers and to absorb bursts. SNS is push-based fan-out to multiple subscribers. EventBridge is an event bus with routing rules, integration with AWS services and SaaS sources, and event-oriented filtering. They can be combined, for example SNS or EventBridge fan-out into separate SQS queues.

IMPORTANT DETAILS

- Standard SQS provides at-least-once delivery, so consumers must be idempotent.
- Use a dead-letter queue with alarms and a replay process.
- Visibility timeout must exceed normal processing time or be extended.
- FIFO is useful only when ordering and deduplication justify lower throughput and tighter constraints.

LIKELY FOLLOW-UPS

- What happens when processing exceeds the visibility timeout?
- How would you handle poison messages?

When would you choose RDS or Aurora versus DynamoDB?

CORE ANSWER

Use a relational database when the domain needs joins, transactions across related records, flexible querying, and strong relational constraints. DynamoDB is effective when access patterns are known, very high scale and low latency are required, and the data can be modelled around partition and sort keys.

IMPORTANT DETAILS

- DynamoDB design starts with access patterns, not normalized entities.
- Avoid hot partitions by distributing high-volume keys.
- Aurora provides relational semantics with managed replication and failover, but schema and query design still matter.
- Use conditional writes for optimistic concurrency in DynamoDB.

LIKELY FOLLOW-UPS

- What is eventual consistency?
- How would you model a many-to-many relation in DynamoDB?
- What causes a database connection storm in serverless workloads?

Explain public subnets, private subnets, NAT gateways, and security groups.

CORE ANSWER

A public subnet has a route to an internet gateway; a resource is internet-reachable only when it also has a public address and permissive controls. Private application subnets do not expose instances directly. A NAT gateway allows outbound internet access from private subnets without accepting unsolicited inbound connections. Security groups are stateful allow rules attached to resources.

IMPORTANT DETAILS

- Use VPC endpoints to reach supported AWS services privately and reduce NAT cost.
- Network ACLs are stateless subnet-level rules and are usually a coarse secondary control.
- Least-privilege security groups should reference other security groups where possible.

LIKELY FOLLOW-UPS

- Why is a database usually placed in private subnets?
- What is the difference between an internet gateway and a NAT gateway?

How do you design resilience around a failing dependency?

CORE ANSWER

Set an explicit timeout shorter than the caller's overall deadline. Retry only transient failures, use exponential backoff with jitter, and cap the number of attempts. Add a circuit breaker when repeated calls are harmful, and isolate resource pools so one dependency cannot exhaust the whole service. Fall back only when a degraded answer is valid for the business.

IMPORTANT DETAILS

- Retries multiply across service layers, so define one responsible retry layer.
- Use idempotency before retrying operations with side effects.
- Queue asynchronous work when immediate completion is not required.
- Expose dependency latency, error rate, saturation, and circuit state as metrics.

LIKELY FOLLOW-UPS

- Why is jitter important?
- What is a retry storm?
- How do you choose a timeout budget?

6. Testing, CI/CD, and observability

Technical interviews often test whether you can deliver and operate software, not only write it.

What should be tested at unit, integration, contract, and end-to-end levels?

CORE ANSWER

Unit tests cover deterministic domain logic quickly. Integration tests verify real boundaries such as a database, queue, framework wiring, or serialization. Contract tests verify that independently deployed services agree on request and response shapes. End-to-end tests validate a small number of critical user journeys through the complete system.

IMPORTANT DETAILS

- Push most business rules below the UI and transport layers so they can be tested cheaply.
- Avoid replacing every dependency with mocks; tests can pass while real integrations fail.
- Use deterministic test data and isolate parallel test runs.

LIKELY FOLLOW-UPS

- What makes a test brittle?
- When is a snapshot test useful?
- How would you test retries and timeouts?

How do blue-green, canary, and rolling deployments differ?

CORE ANSWER

A rolling deployment replaces instances gradually and is resource-efficient, but old and new versions coexist. Blue-green maintains two complete environments and switches traffic, giving fast rollback at higher cost. A canary sends a small percentage of traffic to the new version and expands only when health metrics remain acceptable.

IMPORTANT DETAILS

- Database changes must be backward compatible while versions overlap.
- Automated rollback should use meaningful service-level metrics, not only process health.
- Feature flags can separate deployment from feature release.

LIKELY FOLLOW-UPS

- How would you deploy a breaking database migration safely?
- What metrics would stop a canary rollout?

What is the difference between logs, metrics, and traces?

CORE ANSWER

Logs record discrete events with context. Metrics aggregate numeric behaviour over time and are efficient for dashboards and alerts. Traces show the path and timing of a request across components. Together they answer what failed, how broadly it failed, and where time was spent.

IMPORTANT DETAILS

- Use structured logs with a correlation or trace identifier.
- Prefer service-level indicators such as latency, traffic, errors, and saturation.
- High-cardinality labels can make metrics expensive; keep detailed identifiers in logs and traces.
- Instrument business outcomes as well as infrastructure.

LIKELY FOLLOW-UPS

- What is an SLI, SLO, and error budget?
- How do you trace an asynchronous queue workflow?

How would you debug a production latency regression?

CORE ANSWER

First define the scope: affected endpoints, users, regions, versions, and start time. Compare latency percentiles, error rate, throughput, saturation, deployment events, and dependency timings. Use traces to isolate the slow segment, then validate the suspected cause with logs and resource metrics. Mitigate before performing a perfect root-cause analysis when customer impact is high.

IMPORTANT DETAILS

- Compare p50, p95, and p99 rather than averages alone.
- Look for queue growth, connection-pool exhaustion, event-loop delay, database plans, and downstream timeouts.
- Record a timeline and verify recovery after rollback or mitigation.

LIKELY FOLLOW-UPS

- Why can average latency be misleading?
- What is the difference between root cause and contributing factors?

7. LLMs, MCP, and AI agents

The job description explicitly mentions MCP, agents, orchestration, and intelligent workflows. A practical engineering view is more valuable than AI buzzwords.

When should you use RAG instead of fine-tuning?

CORE ANSWER

Retrieval-augmented generation supplies relevant external knowledge at request time. It is the usual choice when facts change, answers must cite enterprise documents, or access control depends on the user. Fine-tuning changes model behaviour or style and can improve a repeated task pattern, but it is not a reliable way to inject frequently changing factual knowledge.

IMPORTANT DETAILS

- A RAG pipeline needs document parsing, chunking, indexing, retrieval, reranking, and evaluation.
- Apply authorization before or during retrieval, not after generation.
- Measure answer quality with a representative test set, not only subjective demos.

LIKELY FOLLOW-UPS

- How would you choose chunk size?
- What causes poor retrieval even with a strong model?
- When can both RAG and fine-tuning be useful?

What is tool calling, and how should it be implemented safely?

CORE ANSWER

The model selects a declared tool and produces structured arguments. The application validates those arguments, checks authorization and policy, executes the tool, and returns a controlled result to the model. The model should never directly receive unrestricted infrastructure or database access.

IMPORTANT DETAILS

- Treat model output as untrusted input.
- Use strict schemas, allowlists, timeouts, size limits, and idempotency.
- Separate read-only tools from destructive tools and require stronger confirmation or approval for side effects.
- Record tool calls for audit and evaluation without leaking secrets.

LIKELY FOLLOW-UPS

- What happens when the model invents a tool argument?
- How would you prevent duplicate side effects after retries?

What problem does MCP solve?

CORE ANSWER

The Model Context Protocol standardizes how an AI application discovers and invokes external tools, resources, and prompt-like capabilities exposed by an MCP server. It separates the host application, MCP clients, and capability providers, reducing one-off integration code and making tools more portable between AI hosts.

IMPORTANT DETAILS

- MCP is an integration protocol, not an autonomous-agent framework.
- The host still controls trust, user consent, authentication, authorization, and which servers are available.
- Server descriptions and tool outputs are untrusted content and can carry prompt-injection risk.
- Prefer least-privilege, scoped servers rather than one server with broad access.

LIKELY FOLLOW-UPS

- What is the difference between an MCP tool and an MCP resource?
- Where should credentials be stored?
- How would you isolate a third-party MCP server?

When is an agent appropriate, and when is a deterministic workflow better?

CORE ANSWER

Use a deterministic workflow when the steps, branching rules, and success criteria are known. Use an agent when the system must choose among tools or plans under uncertainty and the added flexibility justifies higher cost, latency, and unpredictability. Many production systems are hybrid: deterministic orchestration around a small model-driven decision step.

IMPORTANT DETAILS

- Limit iterations, time, token budget, and tool permissions.
- Persist workflow state so execution can resume safely.
- Use human approval for high-impact actions.
- Prefer explicit state machines for long-running business processes.

LIKELY FOLLOW-UPS

- How do you stop an agent loop?
- What belongs in orchestration code instead of the prompt?

How do you evaluate and observe an AI feature in production?

CORE ANSWER

Build a versioned evaluation set from realistic tasks and expected criteria. Measure task success, groundedness, tool correctness, latency, cost, refusal quality, and safety failures. In production, trace model calls, retrieval results, prompt versions, tool decisions, and final outcomes while applying privacy and retention controls.

IMPORTANT DETAILS

- Use automated graders carefully and calibrate them against human review.
- Separate model quality from retrieval, tool, and orchestration failures.
- Roll out changes gradually and compare versions on the same evaluation set.
- Feedback buttons alone are not a sufficient quality metric.

LIKELY FOLLOW-UPS

- How would you detect prompt injection?
- What data should not be stored in traces?
- How do you test a model upgrade?

8. System design drills

Use these as spoken exercises. Start with requirements and scale, then propose a simple architecture, failure handling, data model, and evolution path.

Design a logistics shipment-tracking platform.

CORE ANSWER

Clarify event sources, update frequency, delivery guarantees, query patterns, customer isolation, and latency expectations. A practical design ingests carrier events through authenticated APIs or queues, validates and normalizes them, stores the current shipment state plus an immutable event history, and publishes updates to downstream consumers. A read API serves current state, while WebSocket or push channels deliver near-real-time changes.

IMPORTANT DETAILS

- Partition by shipment or tenant while avoiding hot keys.
- Make event ingestion idempotent using a carrier event identifier or derived deduplication key.
- Handle out-of-order events with event time, sequence rules, and domain-specific state validation.
- Use a DLQ and replay tooling for invalid or repeatedly failing events.
- Measure ingestion lag, stale shipments, duplicate rate, and notification delivery.

LIKELY FOLLOW-UPS

- How would you support 100 times more events?
- How do you correct a bad carrier event?
- What data is retained and for how long?

Design a pricing recommendation service for logistics operations.

CORE ANSWER

Separate offline feature preparation from online recommendation. Historical shipment, route, capacity, and outcome data can feed batch or streaming feature pipelines. An online API retrieves validated features, calls a model or rules engine, applies business constraints, and returns a recommendation with explanation metadata and a version identifier.

IMPORTANT DETAILS

- Keep a deterministic fallback when the model or feature store is unavailable.
- Record the recommendation, input version, final human decision, and outcome for evaluation.
- Use shadow mode before allowing recommendations to affect prices.
- Prevent data leakage and ensure time-correct training data.
- Monitor drift, acceptance rate, margin impact, latency, and missing-feature rates.

LIKELY FOLLOW-UPS

- Would you calculate synchronously or asynchronously?
- How would you roll back a bad model?
- Where do business rules belong?

Design an AI operations assistant using MCP tools.

CORE ANSWER

The host authenticates the user and exposes only tools allowed for that role. Retrieval provides relevant runbooks and operational context. The model can call read-only diagnostic MCP tools first, such as service health, deployment history, and logs. Any action tool, such as restarting a service or changing configuration, requires strict validation, audit logging, and usually explicit human approval.

IMPORTANT DETAILS

- Keep the incident workflow state outside the model.
- Protect against prompt injection from logs, tickets, and tool descriptions.
- Limit tool scope by environment, service, and action.
- Return structured evidence with every recommendation.
- Evaluate on historical incidents and adversarial inputs before production access.

LIKELY FOLLOW-UPS

- How would you prevent the assistant from restarting the wrong service?
- What happens when two tools disagree?
- How would you support incident handover?

How would you modernize a legacy PHP application gradually?

CORE ANSWER

Use a strangler approach: identify a bounded capability, place a stable routing or API layer in front, and move that capability to the new service while the legacy system continues to handle the rest. Keep data ownership explicit and avoid uncontrolled dual writes. Introduce observability and contract tests before migration so behaviour can be compared.

IMPORTANT DETAILS

- Start with low-risk, high-value boundaries rather than rewriting everything.
- Use an anti-corruption layer to translate legacy concepts into the new domain model.
- For data migration, prefer one authoritative writer and asynchronous replication where possible.
- Define rollback and coexistence before cutting traffic.
- Remove the old path only after usage and correctness are verified.

LIKELY FOLLOW-UPS

- How would you handle shared database tables?
- What is the biggest risk of a big-bang rewrite?
- How do you test behavioural equivalence?

9. Live coding and code review

The goal is not memorizing puzzles. Explain assumptions, choose clear data structures, handle failures, and test edge cases aloud.

How should you approach a live coding task?

CORE ANSWER

Restate the problem and clarify the contract before writing code. Work through a small example, identify the data structure and invariants, then implement the simplest correct solution. Speak about complexity, failure behaviour, and tests as you go instead of trying to produce silent perfect code.

IMPORTANT DETAILS

- Confirm input, output, ordering, mutation, and error expectations.
- Start with correctness; optimize only when constraints require it.
- Run through empty, normal, and adversarial examples.
- When stuck, state the uncertainty and reduce the problem to a smaller step.

LIKELY FOLLOW-UPS

- What is the time and space complexity?
- How would this change for very large input?
- What would you harden for production?

Implement bounded asynchronous concurrency.

CORE ANSWER

A strong solution accepts a list of tasks and a maximum concurrency, starts only that many tasks, and starts the next task whenever one finishes. Define whether results preserve input order and whether one failure stops all processing or is collected per item.

```
async function mapLimit<T, R>(
  items: T[], limit: number, fn: (x: T) => Promise<R>
): Promise<R[]> {
  const out = new Array<R>(items.length);
  let next = 0;
  async function worker() {
    while (true) {
      const i = next++;
      if (i >= items.length) return;
      out[i] = await fn(items[i]);
    }
  }
  await Promise.all(Array.from(
    { length: Math.min(limit, items.length) }, worker
  ));
  return out;
}
```

IMPORTANT DETAILS

- Validate that the limit is at least one.
- Do not create all promises before applying the limit, because the work may start immediately.
- Consider cancellation and a maximum queue size for production use.

LIKELY FOLLOW-UPS

- How would you preserve result order?
- How would you implement fail-fast versus collect-all behaviour?

Review a React component that fetches data in an effect.

CORE ANSWER

Check whether the effect dependencies are complete, whether obsolete requests can overwrite current state, whether loading and error states are coherent, and whether the request should instead be owned by a server-state library. Also inspect behaviour when the component unmounts, the identifier changes quickly, or development Strict Mode starts the effect twice.

IMPORTANT DETAILS

- Avoid storing derived data in a second state variable.
- Abort requests or ignore stale results.
- Do not catch and hide every error; distinguish cancellation from failure.
- Keep the cache key aligned with every input that changes the response.

LIKELY FOLLOW-UPS

- How would you test the race condition?
- Where should retry policy live?

Review an API endpoint that creates a shipment.

CORE ANSWER

Check authentication and tenant authorization, schema validation, request-size limits, idempotency, transaction boundaries, duplicate detection, and error mapping. Confirm that downstream notifications or events cannot be published without the database state being committed. An outbox pattern may be needed when database change and event publication must be coordinated.

IMPORTANT DETAILS

- Do not trust tenant identifiers supplied by the client without policy checks.
- Return a stable resource identifier and appropriate status code.
- Use a unique constraint as the final protection against duplicates.
- Log a correlation identifier, not sensitive request data.

LIKELY FOLLOW-UPS

- When would you return 201, 202, or 409?
- How does the transactional outbox work?

What edge cases should you state during a coding task?

CORE ANSWER

Mention empty input, invalid input, duplicates, ordering, very large input, cancellation, timeouts, partial failure, concurrency, and memory use when relevant. Then state the expected complexity and the behaviour that the caller can rely on. Clear contracts often matter more than a clever implementation.

IMPORTANT DETAILS

- Use representative examples before writing code.
- Name invariants explicitly.
- Add tests for the smallest, normal, and adversarial cases.
- Prefer readable code and explain where production hardening would be added.

LIKELY FOLLOW-UPS

- How would you test a function with time-based behaviour?
- When is $O(n \log n)$ preferable to a complex $O(n)$ solution?

The questions are aligned with the Redspher Full Stack Developer stack and the realistic depth of a senior full-stack technical interview.